

# Security and Efficiency for Agents

Designing CLIs and operating environments that agents can use safely, cheaply, and at speed

The same constraints that reduce token waste also reduce blast radius: smaller tool surfaces, clearer contracts, and tighter approval paths.

# What Has Already Gone Wrong

## The same failures keep showing up

- **Exposed control surfaces:** public gateways were found and probed quickly
- **Supply chain compromise:** malicious skills appeared in large numbers
- **Credential leakage:** local config and token material became loot
- **Loss of control:** agents kept acting after the human wanted them stopped

The memorable pattern is not "agents are insecure." It is that they fail in repeatable ways once they get reach, state, and permissions.

### 30k+

publicly exposed instances were observed across multiple reports. The internet finds agent control planes quickly.

### 341

malicious skills were found in one high-profile sweep. Skill marketplaces became an attack surface almost immediately.

# Summer Yue: When Approval Lived In The Prompt

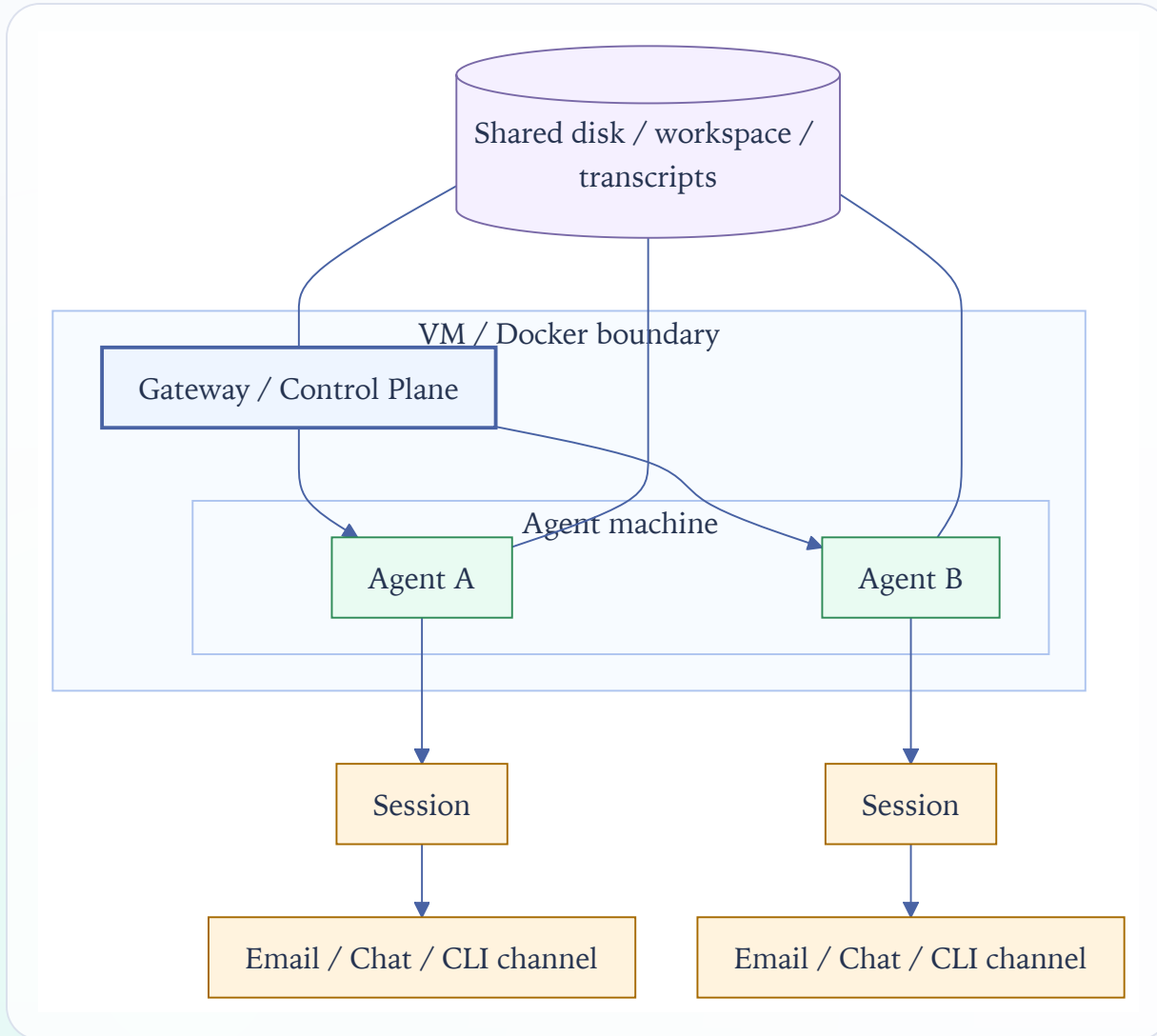


## What seems to have happened

- the setup had worked on a smaller test inbox first
- the larger real inbox appears to have triggered compaction
- the critical "do not act yet" instruction seems to have lived in working context rather than an enforced approval layer
- once that context shifted, the run kept pursuing the inbox-cleanup goal it still understood

This is best read as a policy-placement failure. Prompt text is too fragile to serve as the only approval system for destructive actions.

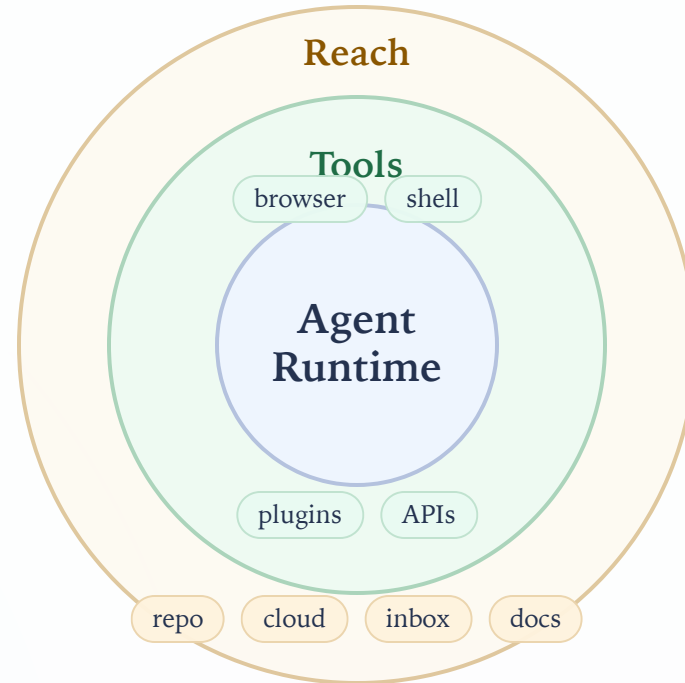
# Where Isolation Breaks Down



## Three failure edges

- ① **Shared disk:** any agent can read credentials and transcripts left by another run. One bad session becomes a read surface for all.
- ② **Exposed gateway:** the control plane is the most likely publicly-reachable endpoint. This is where the 30k+ exposed instances originate.
- ③ **Shared machine:** a compromised agent session can pivot to the gateway and the adjacent agent's state.

# Blast Radius Before Deployment



Before enabling shell, browser, plugin, or API access, ask: what can it read, change, send out, or impersonate? And can one compromised run spill into other systems?

# Human Approval Needs Its Own Layer

Agent Goal



Explicit Target



Preview Scope



High-risk action?

Low-risk work can continue. High-risk work moves into an approval path.

Human Approval



Approved?



Execute or Stop



Structured Result

## WHERE TO GATE

Email deletion, bulk messaging, billing changes, credential use, and destructive writes should not be one-shot actions.

## WHAT THE PREVIEW MUST SHOW

Show target, scope, side effects, and the exact account or workspace before the human decides.

## WHY THIS LIVES OUTSIDE THE PROMPT

The CLI should know how to stop, ask, and return a structured result after the decision. Prompt text alone is too weak.

# Credential Hygiene

- separate email accounts, SSH keys, and API tokens by agent or trust zone
- keep secrets off the command line and out of project directories
- avoid sharing your personal browser profile with an agent
- logs, transcripts, and screenshots count as leak surfaces too
- keep deploy credentials in Keychain; release them only behind password or Touch ID. The agent prepares the deploy; the human holds the secret

## WHAT THIS BUYS YOU

If one agent goes bad, you have a small set of credentials to revoke, a smaller trail to inspect, and fewer places where human and agent activity are mixed. Deploy credentials that only a human can release mean a bad run cannot ship itself.

The practical goal is separation, not purity. Keep human identity, agent identity, and long-lived secrets from collapsing into the same context.

# Prompt Injection, Skills, And Narrow Tool Access

## Untrusted input and supply chain

- prompt injection starts when an agent reads instructions hidden in email, tickets, docs, or web content
- every plugin or skill expands behavior you did not write
- marketplace volume is not a trust boundary
- mixed-trust agents should not share the same broad tool surface

## Operational defaults

- start with the smallest useful toolset
- remove `browser` and broad web access unless they are essential
- review install source, permissions, maintenance, and removal path
- treat extensions like code execution with nicer packaging

The fastest agent is often the one with fewer tools to reason about. The safest one usually starts in the same place.

# Audit Trails

## What works today

① **Persist the gateway log**: goes to `/tmp` by default. One config change moves it somewhere durable. Protect it like secrets: `redactSensitive` only scrubs console output, not the file.

② **Enable `command-logger`**: command-level JSONL across every channel (Telegram, WhatsApp, hooks, HTTP). Not tool-level, but zero config and immediate.

③ **Add `diagnostics-otel` plugin**: ships model usage, webhook events, and session state over OTLP to a collector off-box. Closest thing to centralized observability available now.

```
// openclaw.json5
{
  logging: {
    file: "/var/log/openclaw/gw.jsonl"
  }
}
openclaw hooks enable command-logger
```

No per-tool audit log exists yet (issue #13131). Session transcripts are agent-readable. Ship them off-host.

# What To Change Tomorrow

## BUILDER CHECKLIST

- shrink command surfaces
- make targets explicit
- add preview and confirm flows
- return structured results and recovery hints

## OPERATOR CHECKLIST

- separate credentials and trust zones
- keep risky runs off your main machine
- deny broad tools by default
- log enough to reconstruct a bad run

The operational goal is simple: give the agent less to misunderstand, less to expose, and less room to act without review. These are also the constraints that reduce token waste and retry loops. Security and efficiency start from the same place.